Weekly Progress Report

Refactoring Detection Project - Week 3 progress update for the Monday research supervisor meeting

1. Overview

During Week 3, the team moved from general UI planning into a more evidence-based design process. The main work was completing tool testing, reviewing log files, updating the confidence score into an AI Trust Score, redesigning the UI in Figma using an IntelliJ-style direction, and preparing for implementation work on the UI panel branch.

The Week 3 progress is based on the slide deck, the log review/results document, and the organized raw log evidence collected from the tool runs.

2. Weekly Progress Summary

Area	Progress Made
Tool testing	The team completed tool testing and documented issues from running AntiCopyPaster, including API quota errors, API key validation problems, and testing/environment failures.
Log review	The team reviewed the sample log file shared by Dr. AlOmar and the team log files to identify useful information for the UI and trust score.
UI design	The UI was redesigned on Figma using IntelliJ design patterns and an IntelliJ UI kit so the prototype feels familiar inside the IDE.
AI Trust Score	The score direction was updated based on Dr. AlOmar's feedback: the metric is now an AI Trust Score based on five workflow components, each worth 20%.
Implementation planning	The team started planning how to divide UI panel implementation tasks on GitHub.
Website/files	Weekly files continued to be prepared for the progress website, including slides, report, notes, collaborators, results, and other docs.

3. Meeting Context and Completed Tasks

In the third meeting on 06/01/2026, the team presented the second week slides, discussed issues using the tool, reviewed the UI Panel as the starting point for the UI implementation, and received feedback from Dr. AlOmar about the confidence/trust score direction.

The Week 3 tasks were completed and used to guide this progress update:

- Team members sent summarized weekly progress and completed tasks so the website files could stay updated.
- Errors and warnings from duplicate-method testing were captured and used as examples for UI failure states.
- The sample log file shared by Dr. AlOmar on Discord was reviewed, along with the team logs.
- Juliana and Dhir updated the confidence/trust score direction based on Dr. AlOmar's suggestions.
- The API key issue was identified as a UI/settings problem: validation should accept the AQ. prefix or remove strict prefix validation.
- The team started planning work on the UI panel branch and task division for implementation.

4. Tool Testing and Log Review Results

The log files showed that the tool works as a pipeline. Each stage produces information that can be shown to the developer in the UI instead of hiding the process behind a single AI response.

Pipeline Stage	What the Logs Showed	UI Information to Show
Paste / Setup	Logs include the pasted snippet, file name, model, max attempts, and minimum clone count.	Show that duplicate code was detected after paste, with the file name and selected range.
Detection	Three detection panelists and a curator try to find clones. If the LLM fails, PSI fallback can still find clone ranges.	Show clone type, clone count, line numbers, and whether detection came from LLM or fallback.
Refactoring	Panelists suggest Extract Method candidates and a curator selects the strongest option.	Show the proposed helper method name and a short explanation of what will be replaced.
Usefulness	The checker can reject refactorings that create a helper method but fail to actually replace duplicated code.	Show usefulness pass/fail and a clear reason such as duplicate code still remains.
Compilation	The tool compiles the proposed source in isolation and can distinguish pre-existing project errors from new errors.	Show a separate Compilation status and explain whether errors are new or already existed.
Testing	EvoSuite/test logs can show passed tests, failed tests, missing target class, unsupported Java version, or test generation failure.	Show Tests passed, Tests failed, or Tests unavailable/tool failed as separate states.
Final Decision	The workflow can succeed, fail after retries, or be verified but not applied because the user cancels.	Keep users in control with Apply, Cancel, Open in editor, and View Details actions.

5. Important Issues Found in the Logs

Issue	Evidence From Logs	Design/Implementation Impact
Quota and token-limit failures	Gemini returned error 429 with RESOURCE_EXHAUSTED status and retry delays around 50 seconds. Some runs failed after repeated quota errors.	The UI should show a clear provider error such as AI provider quota exceeded, plus options to retry later, switch model, or check API settings.
LLM unavailable but fallback works	In the RockPaperScissors run, LLM detection failed, but PSI fallback found two clone ranges.	Show a badge such as Detected by PSI fallback or AI unavailable - fallback used, because trust may be lower than a full LLM result.
Incomplete refactoring	In a Main.java run, the usefulness checker rejected a proposal because a helper method was created but duplicate code was not fully replaced.	Usefulness must be a visible validation chip and Apply should stay disabled if the refactoring is not useful.
Compilation and testing are different	Some attempts compiled successfully but tests still failed or were unavailable.	Do not combine compile and test into one status. Show separate Compilation and Test/Behavior states.
Testing tool/environment failure	EvoSuite failed in one run due to issues such as unsupported class file major version 69 and missing generated tests.	Add a Tests unavailable or Testing tool failed state so users do not confuse environment problems with unsafe refactoring.
API key validation	The team identified that newer Google API keys may use an AQ. prefix while the tool expected the older AIzaSy format.	Update settings validation to accept AQ. keys or remove strict prefix validation.

6. Successful Workflow Example

The DeepSeek log showed the clearest successful workflow. It detected three clone occurrences in AprLifecycleListener.java, selected a refactoring candidate, passed usefulness, compiled, tested successfully, and reached workflow success.

Step	Log Result	UI Design Meaning
Detection success	Three clone ranges were found across setSSLEngine, setSSLRandomSeed, and setFIPSMode.	The explanation section should show clone count, methods, and clickable line ranges.

Step	Log Result	UI Design Meaning
Refactoring selected	The curator selected Panelist 2 with confidence 0.95.	The AI Trust area can show selected candidate/refactoring evidence.
Usefulness accepted	Usefulness curator accepted the proposal with confidence 1.0.	Show Usefulness passed as a validation chip.
Compilation ok	Baseline project errors existed, but the proposal compiled in isolation.	Show compilation passed for the proposed change while separating existing project errors.
Testing passed	The log ended with tests passed and workflow success.	Apply can be enabled only after the validation pipeline is safe.
User control	The refactoring was verified but not applied because the user cancelled.	The UI should keep Cancel, Apply, and Open in editor actions visible.

7. AI Trust Score Update

The team changed the metric from a simple Confidence Score to an AI Trust Score. The new score should not rely on model confidence alone. It should be based on workflow evidence that is visible in the logs.

Updated formula:

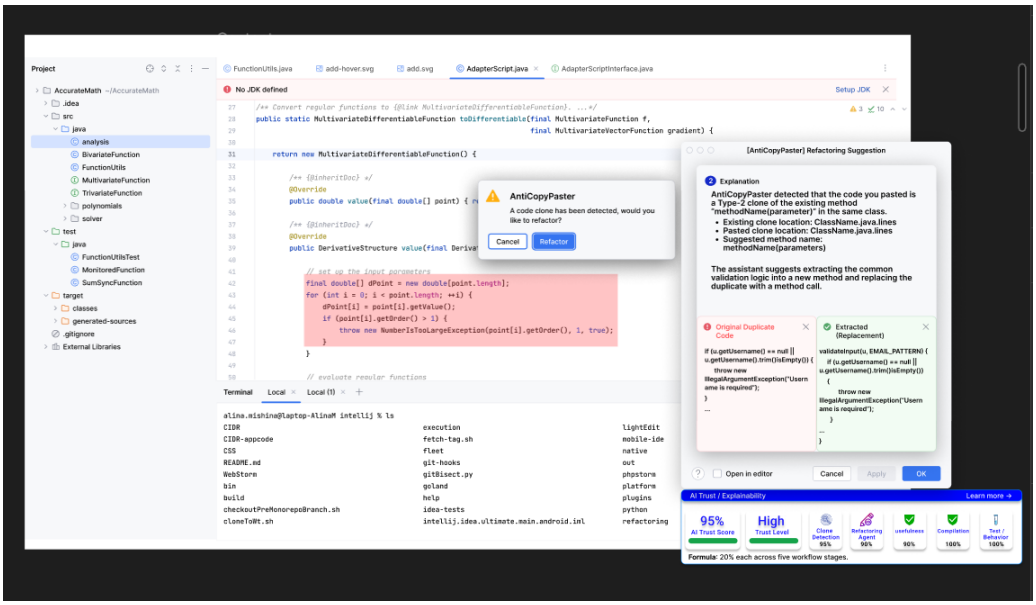
```
AI Trust Score = 20% Clone Detection + 20% Refactoring Agent + 20% Refactoring Usefulness + 20% Compilation Result + 20% Test / Behavior Preservation.
```

This update removes Diff Safety and excludes direct model confidence, following the direction discussed with Dr. AlOmar. If a stage is unavailable because of API quota, testing tool failure, or missing information, the UI should show the score as incomplete or explain which part could not be calculated.

Component	What It Measures	UI Example
Clone Detection	How clearly the tool found real duplicate code and reliable clone ranges.	Clone Detection 95%
Refactoring Agent	Quality of the proposed Extract Method candidate and whether it targets the correct clone.	Refactoring Agent 90%
Refactoring Usefulness	Whether the duplicate code was actually removed/replaced correctly.	Usefulness 90% or failed reason
Compilation Result	Whether the proposed code compiles in isolation without new errors.	Compilation 100%
Test / Behavior Preservation	Whether tests pass or whether testing is unavailable due to environment/tool issues.	Test/Behavior 100%, failed, or unavailable

8. UI Redesign and Connection to Log Evidence

The final UI direction is an IntelliJ-style refactoring suggestion panel. It includes a small confirmation popup, a detailed explanation panel, a diff preview, validation chips, and an AI Trust / Explainability section. This design was chosen because it turns the technical log information into user-facing, understandable feedback.



Log Information	Final UI Feature
File name, clone ranges, pasted snippet	Explanation area with existing clone location, pasted clone location, and line numbers.
Suggested helper method and replacement code	Diff preview with Original Duplicate Code and Extracted Replacement.
Usefulness, compilation, and testing results	AI Trust / Explainability bar with separate validation chips.
Retry attempts and quota failures	Loading state, attempt counter, and clear failure-state messages.
API/provider errors	Settings validation and provider-specific error popups.
Raw log details	Expandable technical details for advanced users instead of cluttering the main panel.

9. UI Ideas Added or Reinforced This Week

UI Idea	Why It Matters
Stage progress indicator	Shows the current step: Detection -> Refactoring -> Usefulness -> Compilation -> Testing. This matches the log structure.
Attempt counter	Logs show attempt 1/3, 2/3, and 3/3. The loading UI should show how many refactoring attempts have been made.
Provider/model status	Helps users understand which AI provider/model is being used and when it is unavailable.
Clear failure-state messages	Different errors need different explanations: quota, API key, usefulness failure, compilation issue, or test tool failure.
AI Trust breakdown	The score should show five separate evidence components instead of a single unexplained percentage.
Expandable technical details	Keeps the main panel simple while still allowing users to inspect logs, panelist decisions, clone ranges, and retry history.
Disable Apply until safe	Apply should stay disabled if usefulness, compilation, or testing is unresolved or failed.
Settings/API key helper	The tool should accept newer AQ. keys or remove strict validation to reduce setup friction.

10. Next Steps

- Continue updating the progress website with weekly files and team progress summaries.
- Begin implementing the redesigned UI on the UI panel branch.
- Divide implementation work so each person owns a specific part of the design.
- Update the API key validation logic so the newer AQ. prefix is accepted or strict validation is removed.
- Continue testing the tool with different models and projects, including alternatives when Gemini quota limits block runs.
- Use future logs to refine the AI Trust Score and make failure states clearer.

11. Conclusion

Overall, Week 3 focused on turning testing results into concrete UI design decisions. The team used the logs to understand what information the tool already produces, what errors developers may face, and how the UI can make AI-generated refactoring suggestions more understandable and trustworthy. The next focus is implementation: using the UI panel branch to begin building the redesigned IntelliJ-style interface and continuing to update the website with weekly progress files.